

УЧЕБЕН МАТЕРИАЛ / ТЕОРЕТИЧЕН КОНСПЕКТ (HANDOUT)

Урок №1: Парадигми за програмиране. Въведение във функционалния модел

Учебен предмет: Функционално програмиране

Специалност: Приложно програмиране

Клас: 12 клас

Име на ученика: **Дата:** **Клас/Група:**

1. Въведение и цели на темата

В съвременната софтуерна разработка функционалният стил вече не е екзотика или чисто академична концепция. Водещи езици и технологии (JavaScript/TypeScript, C# с LINQ, Java със Streams API, Python, Kotlin, Swift, Rust, React) масово интегрират функционални конструкции за обработка на потоци от данни, паралелни изчисления и изграждане на устойчиви софтуерни системи. Разбирането на парадигмите за програмиране ви позволява да избирате най-точния инструмент за всеки конкретен проблем, вместо да разчитате сляпо на един познат шаблон.

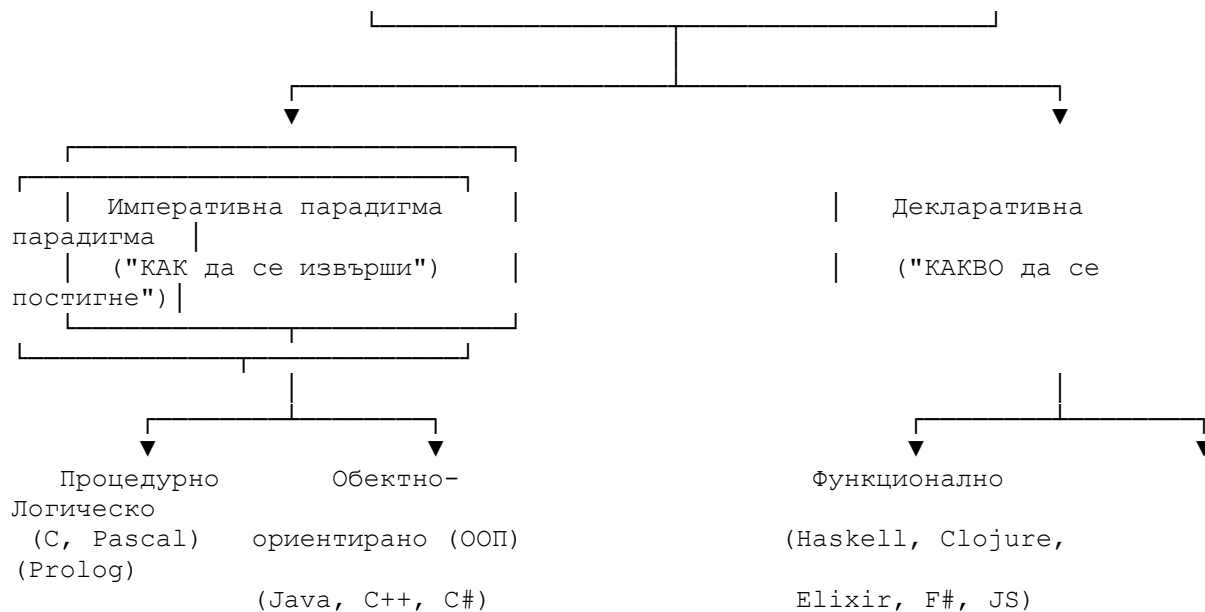
След този урок ще можете да:

- **Разграничавате** фундаментално императивния от декларативния модел чрез анализ на структуриран програмен код;
- **Дефинирате** същността и ключовите стълбове на функционалната парадигма (чисти функции, неизменност, функции от по-висок ред);
- **Разпознавате и анализирате** чисти функции, предвиждайки тяхното поведение без странични ефекти;
- **Оценявате** предимствата на функционалния стил при автоматизирано тестване, дебъгване и поддръжка на софтуер.

2. Синтезиран теоретичен конспект

2.1. Какво е програмна парадигма?

Програмна парадигма (Programming Paradigm) е цялостна философия, концептуален модел и начин на мислене при формулирането и структурирането на програмните решения. Тя определя как програмистът моделира данните, състоянието и изчислителния процес.



2.2. Императивен срещу Декларативен подход

- **Императивен подход (Imperative):** Фокусира се върху **алгоритмичните стъпки (КАК)**. Програмата представлява последователност от команди, променящи състоянието на компютърната памет чрез оператори за присвояване, броячи и цикли (`for`, `while`).
- **Декларативен подход (Declarative):** Фокусира се върху **крайния резултат (КАКВО)**. Програмистът описва математическата зависимост или желаната трансформация на данните, оставяйки нискостепенните детайли по изпълнението на компилатора или средата.

2.3. Основни стълбове на функционалната парадигма

Функционалното програмиране е разновидност на декларативния подход, основана на математическата теория на ламбда смятането (λ -calculus). Основните му стълбове са:

1. Чисти функции (Pure Functions)

Функция се нарича **чиста**, ако отговаря едновременно на две строги условия:

- **Детерминираност (Deterministic):** При подаване на едни и същи входни аргументи, тя **винаги** връща идентичен резултат, независимо от контекста или броя извиквания;
- **Липса на странични ефекти (No Side Effects):** Изпълнението на функцията не променя външно състояние, глобални променливи, файловата система или състоянието на подадените аргументи.

Референциална прозрачност (Referential Transparency): Свойство на израза или функцията, което позволява извикването му да бъде заменено директно с върнатата стойност навсякъде в кода, без това да промени коректността на програмата.

2. Неизменност на данните (Immutability)

След като една структура от данни (променлива, списък, обект) бъде създадена в паметта, нейното състояние **никога не се променя**. Всяка модификация всъщност генерира ново копие с нанесената промяна, запазвайки оригинала непокътнат.

3. Функциите като обекти от първи ред (First-Class Citizens)

Функциите се третират наравно с всички обикновени стойности (числа, низове, булеви):

- Могат да се записват в променливи или константи;
- Могат да се предават като входни аргументи на други функции;
- Могат да се връщат като резултат от изпълнението на друга функция (**Функции от по-висок ред / Higher-Order Functions**).

3. Практически анализ: Императивен vs. Функционален код

Нека решим една и съща практическа задача на JavaScript (мултипарадигмен език, широко използван в практиката):

Задача: По даден масив от числа да се намерят само четните числа, всяко от тях да се повдигне на квадрат и накрая да се изчисли общата им сума.

Вариант А: Императивен стил (стъпка по стъпка с мутация на състоянието)

```
// Входен масив
const numbers = [1, 2, 3, 4, 5, 6];

// Променлива-акумулатор за съхранение на текущото състояние
let sumOfSquares = 0;

// Императивен цикъл с управление на брояча i
for (let i = 0; i < numbers.length; i++) {
  // 1. Проверка на условие
  if (numbers[i] % 2 === 0) {
    // 2. Изчисление на междинна стойност
    const square = numbers[i] * numbers[i];
    // 3. Мутация (промяна) на външната променлива sumOfSquares
    sumOfSquares += square;
  }
}

console.log("Резултат (императивно):", sumOfSquares); // 56
```

- **Анализ:** Кодът описва пълния механизъм на обхождане (`let i = 0; i < ...`). Променливата `sumOfSquares` променя стойността си при всяка итерация. Логиката е силно обвързана в монолитен блок и трудно се тества на части.

Вариант Б: Функционален стил (конвейер от чисти трансформации)

```
// Входен списък
const numbers = [1, 2, 3, 4, 5, 6];

// 1. Чиста предикатна функция за проверка за четност
const isEven = x => x % 2 === 0;

// 2. Чиста функция за повдигане на квадрат
const square = x => x * x;

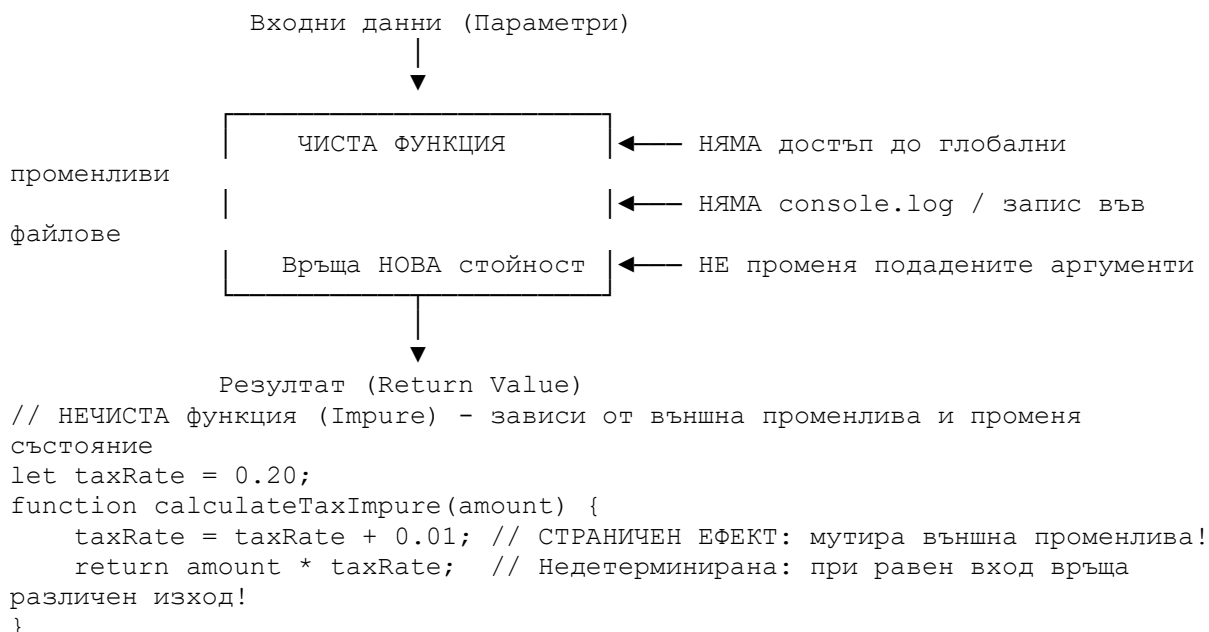
// 3. Чиста функция за сумиране на две стойности
const add = (a, b) => a + b;

// Декларативен конвейер (Pipeline) от чисти функции
const result = numbers
  .filter(isEven)    // Селектира: [2, 4, 6] (върща нов масив)
  .map(square)       // Трансформира: [4, 16, 36] (върща нов масив)
  .reduce(add, 0);    // Редуцира до единична стойност: 56

console.log("Резултат (функционално):", result); // 56
```

- **Анализ:** Няма цикли и индексатори. Няма мутираци променливи. Всяка малка функция (`isEven`, `square`, `add`) е изолирана, чиста, лесна за модулно тестване (Unit Testing) и готова за многократна повторна употреба.

2.4. Как да познаем дали една функция е чиста?



```
// ЧИСТА функция (Pure) - детерминирана и без странични ефекти
function calculateTaxPure(amount, taxRate) {
    return amount * taxRate; // Резултатът зависи ЕДИНСТВЕНО от входните параметри
}
```

4. Внимание: Чести капани и професионални добри практики

❑ Капан / Грешка	💡 Добра практика (Best Practice)	Обяснение
Мутиране на входните аргументи	Връщане на нов масив или копие на обекта	Методи като <code>.sort()</code> , <code>.splice()</code> , <code>.push()</code> променят оригиналния масив. Използвайте функционални алтернативи: <code>map()</code> , <code>filter()</code> , <code>concat()</code> , <code>toSorted()</code> или Spread оператора (<code>[...arr]</code>).
Скрити странични ефекти в чисти функции	Изолиране на I/O логиката в специални слоеве	Дори <code>console.log()</code> или четене на системно време (<code>Date.now()</code> , <code>Math.random()</code>) нарушава чистотата на функцията. Изчислителното ядро трябва да остане 100% свободно от I/O.
Опит за „насилствено“ функционално писане	Използвайте предимствата на парадигмата там, където носят яснота	Не писвайте излишно сложни функционални конструкции за тривиални проверки. Стремете се към максимална четимост, а не към самоцелен синтаксис.

5. Обобщаващ речник на ключовите термини (Cheatsheet)

Термин	Описание / Значение	Кратък синтактичен пример
Парадигма (Paradigm)	Фундаментален модел и стил на мислене при структурирането на код.	<i>Императивна vs. Декларативна</i>
Чиста функция (Pure Function)	Функция без странични ефекти, чийто резултат зависи единствено от нейните аргументи.	<code>const add = (a, b) => a + b;</code>
Неизменност (Immutability)	Принцип, при който данните не се променят след първоначалното им създаване.	<code>const copy = [...orig, 42];</code>
Страничен ефект (Side Effect)	Всяка промяна на състояние извън обхвата на изпълняваната функция (I/O, мутация, база данни).	<code>window.count++;</code> или <code>fs.write(...)</code>

Термин	Описание / Значение	Кратък синтактичен пример
Функция от по-висок ред (Higher-Order Function)	Функция, която приема друга функция като параметър или връща функция като резултат.	<code>arr.filter(isPositive);</code>
Конвейер (Pipeline)	Верижно навързване на функционални трансформации върху поток от данни.	<code>data.filter(...).map(...)</code>

6. Въпроси и задачи за самопроверка

Теоретични въпроси:

1. Коя е ключовата разлика между императивния и декларативния подход при обработката на масиви?
2. Защо чистите функции се тестват значително по-лесно и надеждно в сравнение с нечистите?
3. Може ли функция, съдържаща оператор за извеждане в конзолата (`console.log`), да се счита за напълно чиста? Защо?
4. Какво наричаме „референциална прозрачност“ и защо тя е гарант за безопасно изпълнение?

Практически задачи:

Задача 1 (Анализ и класификация):

Определете кои от следните функции са **чисти** и кои **нечисти**. Обосновете отговора си:

```
// Функция А
let counter = 0;
function increment() {
  counter++;
  return counter;
}

// Функция В
function getCircumference(radius) {
  const PI = 3.14159;
  return 2 * PI * radius;
}

// Функция С
function generateToken(userId) {
  return userId + "_" + Math.random();
}
```

Задача 2 (Рефакториране към функционален стил):

Даден е следният императивен код, който намира най-високата оценка сред одобрените ученици:

```
const grades = [3.50, 5.20, 2.80, 5.80, 4.00, 6.00];
let highestPassingGrade = 0;

for (let i = 0; i < grades.length; i++) {
  if (grades[i] >= 3.00) {
    if (grades[i] > highestPassingGrade) {
      highestPassingGrade = grades[i];
    }
  }
}
console.log(highestPassingGrade); // 6.00
```

Условие: Преобразувайте алгоритъма във функционален стил:

- Дефинирайте чиста предикатна функция `isPassing(grade)`;
- Използвайте вградените функционални методи (напр. `.filter()` и чиста редукция с `Math.max` или `.reduce()`), без да използвате цикли и мутиращи променливи.